

2026-09-22-auto-commit-epoch- ms-producer-identified

Incident 2026-09-22 — Auto- commit epoch-ms producer identified (BOB-068 follow-up)

Revision: 1 **Last modified:** 2026-09-22T09:52:00Z **Reporter:** conductor (qBittorrent-login-repair verification wave, review #9) **Session-under-investigation:** this session, 2026-09-19T17:57:24Z (commit d768373) **Investigator:** independent Fable review subagent, task 9 (“Review the unreviewed auto-commit”) **Class:** unreviewed-commit bypass (§11.4.142 universal-code-review gap) + gate-detection gap (§11.4.201(6) false-null in unattributed-commit-guard.sh) **Terminal state:** PRODUCER IDENTIFIED, GATE FIXED, EXTERNAL TOOL NOT MODIFIED (out of this repo’s scope) **Related:** docs/incidents/2026-08-18-auto-committer-BOB-068-investigation.md (established there was no daemon; this incident identifies the actual human-invoked producer BOB-068’s investigation left open)

1. What happened

Commit d768373 (“Auto-commit 1789833444209”, parent deed8bf) landed on main and was pushed to three remotes (github, upstream, origin) without passing through this project’s own review/commit discipline (§11.4.142, §11.4.234). It mixed six genuine, already-validated fixes from this session with one half-finished, unverified edit (tests/unit/test_no_runtime_service_skips.py, later found RED on main) and silently overwrote committed BOB-095 closure evidence for the fifth

time. Full findings are in the review transcript; this record is scoped to **how the commit reached three remotes with nothing able to refuse it**, per the review’s IMPORTANT-5 finding.

2. The producer, identified

```
~/Projects/project_toolkit/Software-Toolkit/Utils/Git/  
commit.sh:17-27:
```

```
SESSION=$(( $(date +%s%N)/1000000 ) )  
MESSAGE="Auto-commit $SESSION"  
git add .  
git commit -m "$MESSAGE"
```

followed by a push. This script is reachable on PATH as the bare commands `commit` and `cmt` (via `project_toolkit/Upstreamable/commit` and `helix_code/cmt`). It is a **generic, cross-project** convenience wrapper — it has no knowledge of this repo’s `scripts/commit-push-all.sh` mandate (§11.4.234), runs `git add .` unconditionally, and stages/commits/pushes whatever the current working tree happens to hold.

3. Provenance — not a daemon, an interactive invocation

- Local HEAD reflog: `commit: Auto-commit 1789833444209` at 2026-09-19T17:57:24+02:00, created in **this** checkout.
- `github/main` “update by push” at 17:57:27; `upstream/main` at 17:57:29; `origin/main` fast-forward fetch at 17:59:10 — all three remotes updated within ~2 minutes of the local commit, consistent with the same push reaching each configured remote in sequence, not a separate daemon per remote.
- `journalctl`: an interactive SSH session for `milosvasic` from a LAN host (`fe80::8817:71ff:fe60:49c9%enp5s0`) opened 15:19:52, disconnected 17:58:18 — **54 seconds after the push landed**. This is consistent with an operator-driven shell session, not a background timer or daemon (BOB-068’s original investigation already ruled out a daemon; this confirms it for this specific instance).
- The same script produced a sibling commit in a different checkout (`~/Projects/vasic, commit 7ee369d, “Auto-commit 1789823284278”`) at 15:08:05, inside the same SSH session window — i.e. the operator (or an agent acting in an interactive shell on the operator’s behalf)

ran the generic `commit/cmt` shorthand across multiple project checkouts in one sitting, and this repo was one of them.

Verdict: this is the identified producer BOB-068's Phase-1 investigation left open as “not a daemon, race condition or operator-shell toolkit sweep” — resolved here to the latter: an operator-shell invocation of a cross-project generic tool, not a race condition and not a background process.

4. Why nothing refused it

- `core.hooksPath` is unset in this checkout; `.git/hooks/` is empty. No git hook could have intercepted the commit or the push (§11.4.234's mandate — hooks never gate routine commit/push — was already satisfied here, just not in the direction that would have caught this).
- `scripts/hooks/unattributed-commit-guard.sh` exists and is designed to catch exactly this shape (`^Auto-commit$` plus similar templated patterns), but its pattern set did not include the epoch-millisecond-suffixed variant this script actually emits. Confirmed by direct RED/GREEN reproduction (§5).
- This guard is not itself wired into any automatic commit-time hook (consistent with §11.4.234 — it is meant to run as an explicit stage of the sanctioned `scripts/commit-push-all.sh` wrapper), so even a correct pattern would only have caught this **retroactively**, on a subsequent sweep — not prevented the push.

5. Fix applied (this repo only)

`scripts/hooks/unattributed-commit-guard.sh`: added `^[0-9]+$` to `BARE_PATTERNS`, alongside the pre-existing `^Auto-commit$`.

RED (before): `--range deed8bf..d768373` → OK — no unattributed bare/templated commit (`rc=0`) — confirmed miss. Control needle: `--range 0e61779~1..0e61779` (the original bare-shape commit) → fires, `rc=1` — proved the instrument was not blind, only under-matched.

GREEN (after): `--range deed8bf..d768373` → fires, names `d768373` by hash and subject, `rc=1`. Both control needles re-run and hold: the original bare shape still fires; a genuinely attributed commit (`deed8bf` itself) still does not fire. The script's own `--self-test` (golden-good/golden-bad/false-positive oracle) still passes after the change.

6. What was deliberately NOT done

`~/Projects/project_toolkit` is a **separate repository**, outside this checkout and outside this task's authorization. §11.4.28 (submodule/dependency decoupling) and plain scope discipline both argue against unilaterally editing another project's tooling because of an incident observed here. Two possible directions exist and are recorded as an **owed operator decision**, not silently picked:

1. Make `commit.sh` repo-aware (detect a `scripts/commit-push-all.sh` in the target checkout and delegate to it, or refuse with a pointer) — a change to `project_toolkit` itself, needing that project's own review discipline.
2. Leave `commit/cmt` as an intentionally generic, project-agnostic convenience for repos with no stricter mandate, and rely on **this repo's own** guard (now fixed) to catch and report an unreviewed sweep after the fact, per the detection-not- prevention model §11.4.234 already establishes for hooks in this project.

Recommendation, not a decision made on the operator's behalf: (1) is the more durable fix if `commit/cmt` is routinely invoked across multiple Helix-governed checkouts; (2) requires no cross-repo change and is what this incident's §5 fix already delivers.

7. Cross-reference

BOB-068's original investigation (`docs/incidents/2026-08-18-auto-committer-BOB-068-investigation.md`) should be updated to reference this record as the resolution of its open "operator- shell toolkit sweep" hypothesis for at least this instance.